ChatGPT said:

Let's break this down **line by line**, starting with what the program does overall, then explaining each part in simple terms.

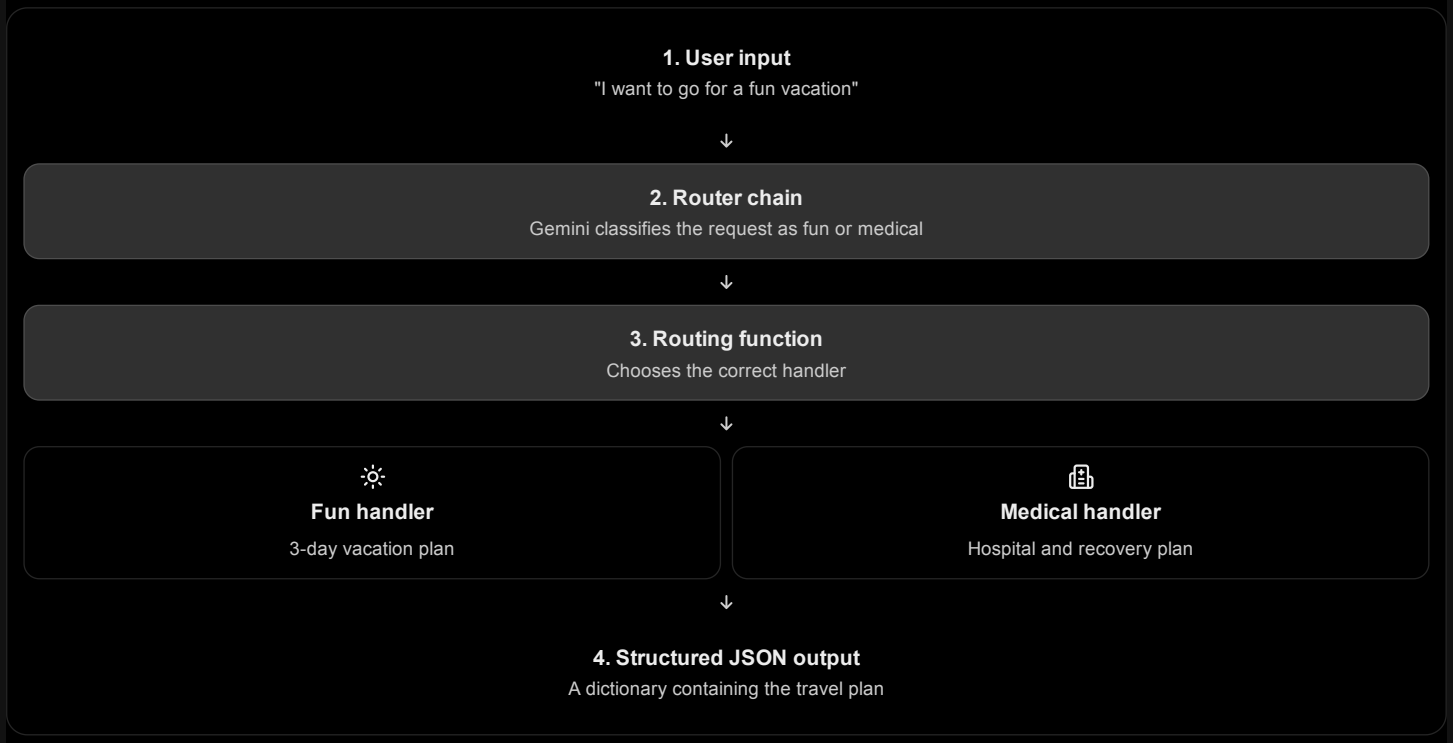
Your program is building an **AI travel router** using LangChain and Gemini. It decides whether a user wants a fun vacation or medical travel, then generates a travel plan in a structured JSON format.

1. Understand the big picture first

Imagine a user enters:

> "I want to go for a fun vacation."

Your application follows these steps:



The most important concepts to understand are **Pydantic models, prompt templates, parsers, chains, routing, and** `.invoke()`.

2. Imports — what each library does

Your first lines:

Python

```
import os
from pydantic import BaseModel, Field
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.runnables import RunnableLambda
from typing import Literal
from dotenv import load_dotenv
```

Import	Purpose
`import os`	Accesses operating-system features, including environment variables.
`BaseModel`	Creates a Pydantic model that defines the expected structure of data.
`Field`	Defines requirements and descriptions for individual fields in a model.
`ChatGoogleGenerativeAI`	Connects LangChain to Google's Gemini chat models.
`ChatPromptTemplate`	Creates reusable prompts with placeholders.

`JsonOutputParser`	Converts the LLM's response into structured Python data and uses formatting instructions.
`RunnableLambda`	Wraps a normal Python function so it can be used inside a LangChain chain.
`Literal`	Restricts a value to specific allowed options.
`load_dotenv`	Loads environment variables from a `.env` file.

Why do we need these?

Suppose Gemini responds with:

```JSON
{
  "travel_type": "fun"
}
```

You want your program to receive a Python dictionary instead of an unprocessed response containing text and formatting. The parser helps with that.

You also want to prevent the router from returning arbitrary categories like `"shopping"` or `"education"`. Pydantic's model and `Literal` define the expected structure and allowed values.

---

3. Loading environment variables

```Python
load_dotenv()
```

This tells Python to look for a `.env` file and load its environment variables into the process environment.

For example, your `.env` file could contain:

```dotenv
GOOGLE_API_KEY=your_actual_api_key
```

Then this line later in your code:

```Python
os.getenv("GOOGLE_API_KEY")
```

can retrieve the key without writing the secret directly in your Python code.

Important: Add `.env` to `.gitignore` so you don't accidentally upload your API key to GitHub.

4. Initializing Gemini

```Python
llm = ChatGoogleGenerativeAI(
```

```
    model="gemini-3.8-flash",
    google_api_key=os.getenv("GOOGLE_API_KEY"),
    temperature=0
)
```

Let's look at each line.

- `llm =` stores the configured language model in a variable named `llm`. You will reuse this variable for all three AI tasks.
- `ChatGoogleGenerativeAI(...)` creates the LangChain interface to a Google Gemini model.
- `model="gemini-3.8-flash"` specifies the model identifier. Verify that this exact model ID is supported by your installed integration and Google account; an unsupported model ID will fail when you call it.
- `google_api_key=os.getenv("GOOGLE_API_KEY")` retrieves your API key from the environment.
- `temperature=0` requests low randomness in the model's responses.

Why set the temperature to zero? Your application expects predictable classifications and structured output. Lower randomness can help, although it does **not** guarantee identical answers or valid JSON every time.

Also, `llm` is not the model's response. It is the configured object you call when you want the model to generate a response.

5. `RouterInput` — defining the router's output

Python

```
class RouterInput(BaseModel):
    travel_type: Literal["fun", "medical"] = Field(
        ..., description="Type of travel (e.g., 'fun', 'medical')"
    )
```

Let's break it down.

`class RouterInput(BaseModel):`

- `class` defines a new Python class.
- `RouterInput` is the name of your class.
- `(BaseModel)` means this class inherits from Pydantic's `BaseModel`.

You're creating a **schema** that describes what the router's output should look like.

Think of a schema as a form that specifies which fields are allowed and what type of data each field must contain.

`travel_type: Literal["fun", "medical"]`

This declares a field called `travel_type`.

The `Literal` type restricts the field to two permitted values:

- `"fun"`
- `"medical"`

For example, this is valid:

Python

```
{"travel_type": "fun"}
```

But `"shopping"` is not a valid value for this schema.

`= Field(...)`

`Field()` adds validation requirements and metadata to the field.

The `...` means the field is **required**. It must be provided.

`description="Type of travel ..."`

This describes the field. LangChain can use this description when generating formatting instructions for the AI.

The important distinction: `RouterInput` does not classify the request itself. Gemini does the classification. `RouterInput` defines what the classification result should look like.

---

6. `FunTravelPlan` — defining a fun vacation plan

Python

```python
class FunTravelPlan(BaseModel):
    city: str = Field(
        ..., description="The recommended city for fun travel"
    )
    day1: str = Field(
        ..., description="Plan for day 1 - main fun activity or area"
    )
    day2: str = Field(
        ..., description="Plan for day 2 - main fun activity or area"
    )
    day3: str = Field(
        ..., description="Plan for day 3 - main fun activity or area"
    )
```

You are defining a second schema, this time for the final fun travel plan.

- `city: str` means `city` should be a string.
- `day1: str` means the first day's plan should be a string.
- `day2: str` means the second day's plan should be a string.
- `day3: str` means the third day's plan should be a string.
- Each `Field(...)` makes that field required and provides a description.

For example, a valid output would look like this:

JSON

```json
{
  "city": "Paris",
  "day1": "Visit the Eiffel Tower and explore the city",
  "day2": "Explore museums and enjoy French cuisine",
  "day3": "Visit Montmartre and walk along the Seine"
}
```

This schema gives Gemini a clear target structure. It does not guarantee that the recommended activities are factually accurate or currently available.

Notice that you use `str` for each day's plan. This means each day can contain a whole sentence or several activities, rather than requiring a separate field for every activity.

---

7. `MedicalTravelPlan` — defining a medical travel plan

Python

```python
class MedicalTravelPlan(BaseModel):
    city: str = Field(
        ..., description="The recommended city for medical travel"
    )
    hospital: str = Field(
        ..., description="Top hospital or clinic in the city"
    )
    day1: str = Field(
        ..., description="Plan for day 1 - focus on medical appointments or procedures"
```

```
    )
    day2: str = Field(
        ..., description="Plan for day 2 - recovery and wellness activities"
    )
    day3: str = Field(
        ..., description="Plan for day 3 - optional sightseeing or relaxation"
    )
```

This works exactly like the fun travel schema, but it defines a different output.

The difference is that `MedicalTravelPlan` includes a `hospital` field, whereas `FunTravelPlan` does not.

This is useful because **different types of requests can require different output structures**.

For example, a medical travel plan might contain:

JSON

```
{
  "city": "Example City",
  "hospital": "Example Hospital",
  "day1": "Attend a consultation",
  "day2": "Follow the clinician's recovery instructions",
  "day3": "Rest or enjoy light sightseeing if medically appropriate"
}
```

This is only an illustration of the structure, not a medical recommendation. Actual treatment and recovery plans should come from qualified healthcare professionals.

8. Creating the output parsers

Python

```
router_input_parser = JsonOutputParser(
    pydantic_object=RouterInput
)

fun_parser = JsonOutputParser(
    pydantic_object=FunTravelPlan
)

medical_parser = JsonOutputParser(
    pydantic_object=MedicalTravelPlan
)
```

You have three schemas, so you create three parsers.

Let's focus on the first:

Python

```
router_input_parser = JsonOutputParser(
    pydantic_object=RouterInput
)
```

- `JsonOutputParser(...)` creates a parser for JSON output.
- `pydantic_object=RouterInput` tells the parser which schema describes the expected response.
- `router_input_parser =` saves that parser so you can use it later.

The other two parsers work the same way, but use different schemas.

What does a parser actually do?

Imagine Gemini returns a response containing JSON text:

```json
{
  "travel_type": "fun"
}
```

The parser converts that response into a Python dictionary:

```python
{"travel_type": "fun"}
```

It also uses the Pydantic schema to help describe the expected output format.

One subtle but important detail: `JsonOutputParser` is not the same as full Pydantic validation. If you need strict validation against the schema, consider `PydanticOutputParser` or explicitly validate the parsed result with the Pydantic model.

9. Creating the prompt templates

A prompt is the instruction you give to the AI. A prompt template lets you reuse the same instruction with different inputs.

Router prompt

```python
router_prompt_b = ChatPromptTemplate.from_template(
    "A user wants to travel. Determine the type of travel based on their request: {travel_goal}. Use the format instruction.\n\n[format_instructions]"
)
```

Let's examine the parts.

`ChatPromptTemplate.from_template(...)`

Creates a reusable prompt template.

`{travel_goal}`

This is a placeholder. Later, you'll supply the user's actual travel request.

For example, if you pass:

```python
{"travel_goal": "I want to go for a fun vacation"}
```

the resulting prompt contains:

> A user wants to travel. Determine the type of travel based on their request: I want to go for a fun vacation.

`\n\n`

This inserts two newline characters, leaving a blank line between the instructions.

`[format_instructions]`

This is intended to tell Gemini what output format to follow, but there is a bug here: square brackets do not create a LangChain template variable.

The correct placeholder is:

```python
{format_instructions}
```

And the formatting instructions must actually be supplied to the template. We'll fix that shortly.

Fun travel prompt

```python
fun_prompt_b = ChatPromptTemplate.from_template(
    "A user wants a fun travel experience for this goal: {travel_goal}. Determine the best city and outline a 3-day fun travel plan. Use the format ins
)
```

This follows the same pattern, but asks Gemini to generate a three-day fun vacation plan.

It has two intended inputs:

- `travel_goal`: what the user wants to do.
- `format_instructions`: instructions describing the expected JSON structure.

Medical travel prompt

```python
medical_prompt_b = ChatPromptTemplate.from_template(
    "A user wants to travel for medical reasons for this goal: {travel_goal}. Determine a suitable city, top hospital, and outline a 3-day medical trav
)
```

This asks Gemini to create a medical travel plan instead.

The important concept is that you are **reusing one LLM with three different prompts**, rather than creating three separate AI models.

---

10. Building the router chain

```python
router_chain = router_prompt_b | llm | router_input_parser
```

This is one of the most important lines in the program.

The `|` operator is part of LangChain Expression Language (LCEL). It connects components into a pipeline.

`router_prompt_b`
Builds the prompt using the user's travel goal

↓

`llm`
Gemini determines the travel type

↓

`router_input_parser`
Parses the JSON response into a Python dictionary

You can think of it as:

```python
output = router_prompt_b.invoke(input_data)
output = llm.invoke(output)
output = router_input_parser.invoke(output)
```

This is a conceptual illustration of the pipeline; LCEL handles the intermediate calls for you.

The complete chain is stored in `router_chain`, ready to be executed later.

For example, the intended result of invoking the chain is:

```python
{"travel_type": "fun"}
```

However, your current router prompt has the formatting-instruction bug described earlier. You should fix that before relying on this result.

11. The `handle_fun()` function

```python
def handle_fun(travel_goal: str) -> dict:
```

This defines a Python function named `handle_fun`.

- `travel_goal: str` indicates that the function expects a string argument.
- `-> dict` indicates that the function is intended to return a dictionary.
- `def` starts the function definition.

For example:

```python
handle_fun("I want a beach vacation")
```

The function receives `"I want a beach vacation"` as `travel_goal`.

Next line

```python
fun_prompt = fun_prompt_b.partial(
    format_instructions=fun_parser.get_format_instructions()
)
```

This line combines two concepts.

First:

```python
fun_parser.get_format_instructions()
```

This generates instructions describing the JSON format expected by your fun travel parser.

Those instructions are derived from the `FunTravelPlan` schema.

Second:

```python
fun_prompt_b.partial(...)
```

`partial()` fills in a template variable in advance.

In this case, it supplies `format_instructions` now, while leaving `travel_goal` to be supplied later.

Think of it as preparing the prompt template before the actual user request arrives.

Next line

```python
fun_chain = fun_prompt | llm | fun_parser
```

This creates the complete fun travel pipeline:

1. `fun_prompt` prepares the prompt.
2. `llm` generates the travel plan.
3. `fun_parser` parses the result into a Python dictionary.

Final line

```python
```

```
return fun_chain.invoke({"travel_goal": travel_goal})
```

This executes the chain.

- `.invoke()` runs the pipeline with the provided input.
- `{"travel_goal": travel_goal}` supplies the user's travel goal in dictionary form.
- `return` sends the parsed result back to whichever part of the program called the function.

For example, if `travel_goal` is `"I want a beach vacation"`, the output might resemble:

Python

```
{
    "city": "Barcelona",
    "day1": "Relax at the beach",
    "day2": "Explore the city",
    "day3": "Visit local attractions"
}
```

These are illustrative values; Gemini chooses the actual response.

12. The `handle_medical()` function

Python

```
def handle_medical(travel_goal: str) -> dict:
    medical_prompt = medical_prompt_b.partial(
        format_instructions=medical_parser.get_format_instructions()
    )
    medical_chain = medical_prompt | llm | medical_parser
    return medical_chain.invoke({"travel_goal": travel_goal})
```

This function works almost identically to `handle_fun()`.

The difference is which prompt and parser it uses:

Fun handler	Medical handler
`fun_prompt_b`	`medical_prompt_b`
`fun_parser`	`medical_parser`
Returns a city and three days of fun activities	Returns a city, hospital, and three days of medical travel planning

Why use separate functions?

Because the router can select the correct function without needing to know the details of how each plan is generated.

This is an example of **separation of concerns**: each function has a specific responsibility.

13. The routing function — `route()`

This is where your program decides which handler to execute.

Python

```
def route(output: dict) -> dict:
```

The function expects a dictionary called `output`, and it is intended to return a dictionary.

First line inside the function

```python
travel_type = output.get("travel_type", "").lower()
```

Let's break this down.

`output.get("travel_type", "")`

Looks for the key `"travel_type"` in the dictionary.

If the key exists, it returns its value. If the key is missing, it returns an empty string (`""`).

For example:

```python
output = {"travel_type": "fun"}
```

Then:

```python
output.get("travel_type", "")
```

returns:

```python
"fun"
```

`.lower()`

Converts the string to lowercase.

For example:

```python
"FUN".lower()
```

returns `"fun"`.

This helps the routing logic handle capitalization differences.

Second line

```python
travel_goal = output.get("travel_goal", "")
```

Retrieves the original travel request from the dictionary.

For example:

```python
{
    "travel_type": "fun",
    "travel_goal": "I want a beach vacation"
}
```

The variable `travel_goal` becomes `"I want a beach vacation"`.

The `if` statement

```python
if "fun" in travel_type:
    return handle_fun(travel_goal)
```

If the travel type contains `"fun"`, the function calls `handle_fun()` and returns its result.

The `return` means the function ends once it returns that result.

The `elif` statement

```python
elif "medical" in travel_type:
    return handle_medical(travel_goal)
```

If the first condition isn't met, Python checks whether the travel type contains `"medical"`.

If it does, the medical handler is called.

The `else` statement

```python
else:
    return {
        "error": "Sorry, I can only handle fun or medical travel plans."
```

If neither condition is satisfied, the function returns an error dictionary.

This is a fallback for unsupported travel types.

One improvement: since the router schema already restricts the category, exact comparisons such as `travel_type == "fun"` are clearer than checking whether `"fun"` appears anywhere in the string.

14. Building the complete application

Now we reach the final pipeline:

```python
router_app = {
    "travel_type": router_chain,
    "travel_goal": lambda x: x["travel_goal"]
} | RunnableLambda(route)
```

This is probably the trickiest part of the code, so let's take it slowly.

Part A: The dictionary

```python
{
    "travel_type": router_chain,
    "travel_goal": lambda x: x["travel_goal"]
}
```

In LCEL, a dictionary of runnable components can be used as a parallel mapping. Both entries receive the same input, and their results are combined into a new dictionary.

Suppose the initial input is:

```python
{"travel_goal": "I want to go for a fun vacation"}
```

First entry:

```python
"travel_type": router_chain
```

This runs the router chain using the original input. Its result should be:

```python
{"travel_type": "fun"}
```

Notice something important: the router chain returns a dictionary, not just the string `"fun"`.

Second entry:

```python
"travel_goal": lambda x: x["travel_goal"]
```

What is `lambda`?

A `lambda` is a short, anonymous function.

This:

```python
lambda x: x["travel_goal"]
```

is equivalent to:

```python
def get_travel_goal(x):
    return x["travel_goal"]
```

It receives the original input dictionary and extracts the user's travel goal.

Its result is:

```python
"I want to go for a fun vacation"
```

Part B: Combining the results

The dictionary mapping produces a structure similar to this:

```python
{
    "travel_type": {
        "travel_type": "fun"
    },
    "travel_goal": "I want to go for a fun vacation"
}
```

Then this part runs:

```python
| RunnableLambda(route)
```

`RunnableLambda(route)` wraps your ordinary Python `route()` function so that LangChain can execute it as part of the pipeline.

But there is a bug here. Your `route()` function expects `output["travel_type"]` to be a string, but the mapping passes a dictionary. Calling `.lower()` on that dictionary causes an `AttributeError`.

We'll fix that below.

15. The entry point of your Python program

```python
if __name__ == "__main__":
    result = router_app.invoke(
        {"travel_goal": "I want to go for a fun vacation"}
    )
    print(result)
```

`if __name__ == "__main__":`

This is a Python convention.

It means: execute the indented code only when this file is run directly, rather than when it is imported as a module into another Python file.

`result = router_app.invoke(...)`

This runs your complete application pipeline.

The input dictionary contains the user's travel goal.

The program should then:

1. Classify the request.
2. Select the fun or medical handler.
3. Generate the appropriate travel plan.
4. Return the result.

`print(result)`

Prints the returned Python dictionary to the terminal.

For example, the output might be:

```python
{
    "city": "Barcelona",
    "day1": "Visit the beach",
    "day2": "Explore the city",
    "day3": "Enjoy local attractions"
}
```

---

16. The important bugs to fix

Your overall design is reasonable, but there are two key issues that will prevent the program from working as intended.

Bug 1: The format instructions are not inserted correctly

Your prompts contain:

```
Python

[format_instructions]
```

But LangChain expects:

```
Python

{format_instructions}
```

Also, you never provide the formatting instructions to `router_prompt_b`, unlike your fun and medical prompts.

Fix it like this:

```
Python

router_prompt_b = ChatPromptTemplate.from_template(
    """A user wants to travel.
    Determine the type of travel based on their request: {travel_goal}.
    Follow these format instructions:
    {format_instructions}"""
)

router_prompt = router_prompt_b.partial(
    format_instructions=router_input_parser.get_format_instructions()
)

router_chain = router_prompt | llm | router_input_parser
```

Now the router receives the expected JSON format instructions too.

Bug 2: The router returns a dictionary inside another dictionary

You can fix the `route()` function to extract the nested classification:

```
Python

def route(output: dict) -> dict:
    travel_type = output["travel_type"]["travel_type"].lower()
    travel_goal = output["travel_goal"]

    if travel_type == "fun":
        return handle_fun(travel_goal)
    elif travel_type == "medical":
        return handle_medical(travel_goal)
    else:
        return {"error": "Unsupported travel type"}
```

This works with your existing dictionary mapping, assuming the router chain returns the expected parsed dictionary.

An alternative is to simplify the mapping so that `travel_type` is already a string:

Python

```python
router_app = {
    "travel_type": router_chain | RunnableLambda(
        lambda result: result["travel_type"]
    ),
    "travel_goal": lambda x: x["travel_goal"]
} | RunnableLambda(route)
```

With this version, your original `route()` function can read `output["travel_type"]` as a string.

17. Test your understanding

Question 1: What is the purpose of `FunTravelPlan`?

- ○ It generates the travel plan by itself.
- ◉ It defines the expected structure of the travel plan.
- ○ It connects Python to Gemini.

Question 2: What does `prompt | llm | parser` mean?

- ◉ The prompt is prepared, sent to the model, and the response is parsed.
- ○ Three models run at the same time.
- ○ The parser sends the prompt to the model.

Question 3: Why do we use `RunnableLambda(route)`?

- ○ To create a new AI model.
- ○ To convert the user's input into JSON automatically.
- ◉ To use an ordinary Python function inside an LCEL pipeline.

Your results

✓ Question 1: Correct!

✓ Question 2: Correct!

✓ Question 3: Correct!

Try again

The main takeaway: Your program separates classification from generation. Gemini first decides *which type of travel the user wants*, your routing function chooses the appropriate handler, and a second LLM call generates a plan using the relevant schema. This pattern is useful for building AI systems that route requests to different specialized workflows.